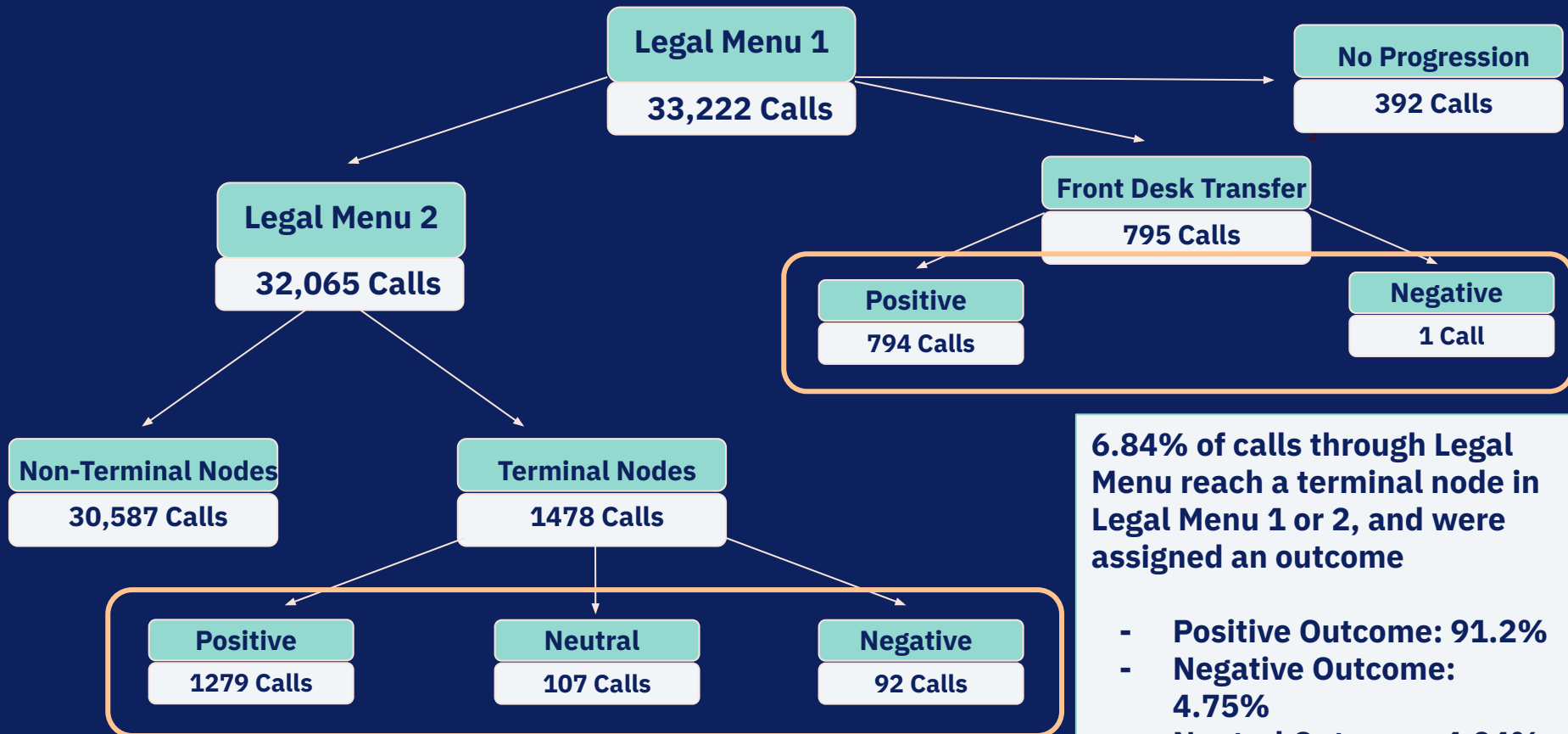


LEGAL AID CHICAGO



Legal Menu Outcomes



6.84% of calls through Legal Menu reach a terminal node in Legal Menu 1 or 2, and were assigned an outcome

- Positive Outcome: 91.2%
- Negative Outcome: 4.75%
- Neutral Outcome: 4.04%

Code for identifying each outcome can be found in appendix

Legal Menu Outcomes

Legal Menu 1

Legal Menu 2

**HIV, Family,
Housing, Benefits,
Employment, and
Immigration
Menus do not
terminate within
Legal Menu**

ADAPT

Total Calls: 68
By Outcome:
- Positive: 60
- Negative: 8

Consumer

Total Calls: 701
By Outcome:
- Positive: 617
- Negative: 84

Criminal Records

Total Calls: 709
By Outcome:
- Positive: 602
- Neutral: 107

Front Desk Transfer

Total Calls: 795
By Outcome:
- Positive: 794
- Neutral: 1

Legal Menu 2 Outcomes

Overall Outcomes for Legal Menu 2:

- **Positive: 86.5%**
- **Negative: 7.24%**
- **Neutral: 6.22%**

By Sub-Menu

ADAPT

Positive Outcomes
88%

Negative Outcomes
12%

Consumer

Positive Outcomes
88%

Negative Outcomes
12%

Criminal Records

Positive Outcomes
85%

Neutral Outcomes
15%

Immigration Menu Outcomes

Prompt: Immigration Menu

1055 calls

Queue 1: Trafficking Voicemail Transfer

43 calls

By Outcome:

- Positive: **43**
- Neutral: 0
- Negative: 0

Queue 2: Immigration / Immigration SP

Group suboptions:

Visa or Crime Victim / Asylum / Immigration
Court / Naturalization

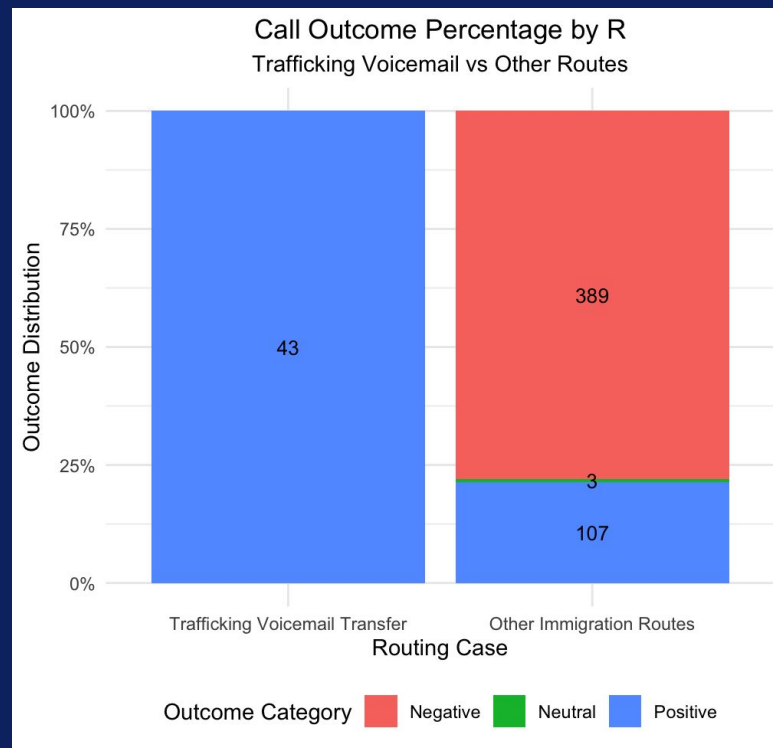
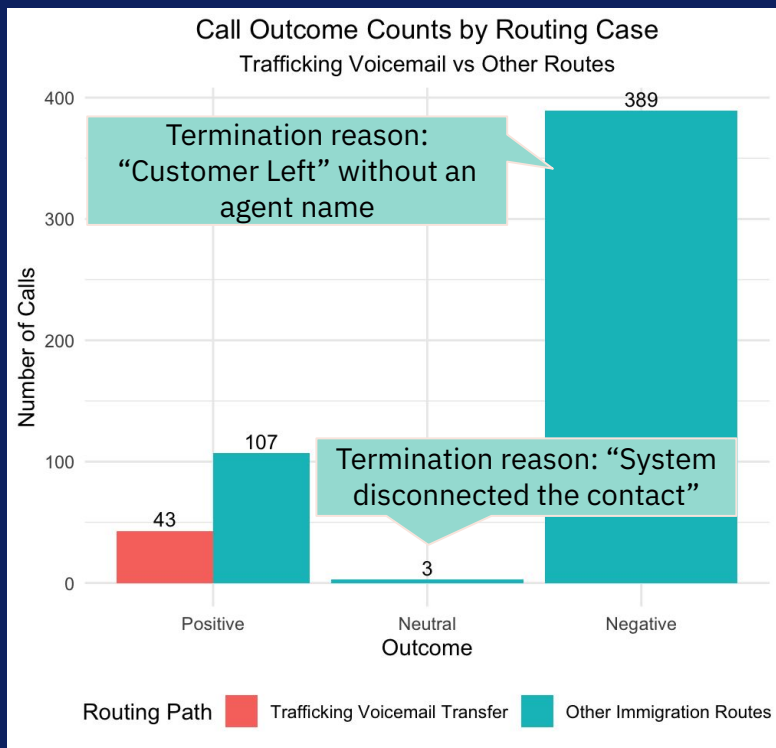
1012 calls

By Outcome:

- Positive: **107**
- Neutral: **3**
- Negative: **389**

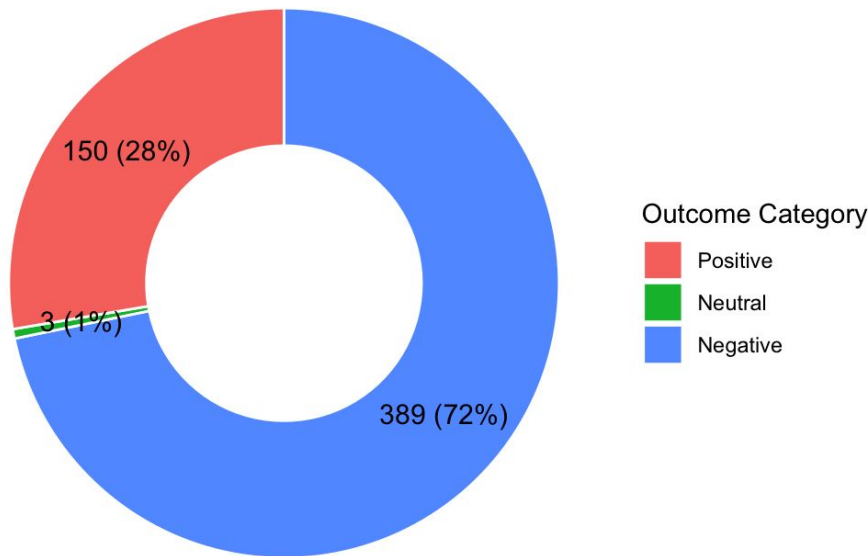
Dropped **513**
NA cases

Immigration Menu Outcomes



Immigration Menu Outcomes

Overall Immigration Call Outcomes



Takeaway:

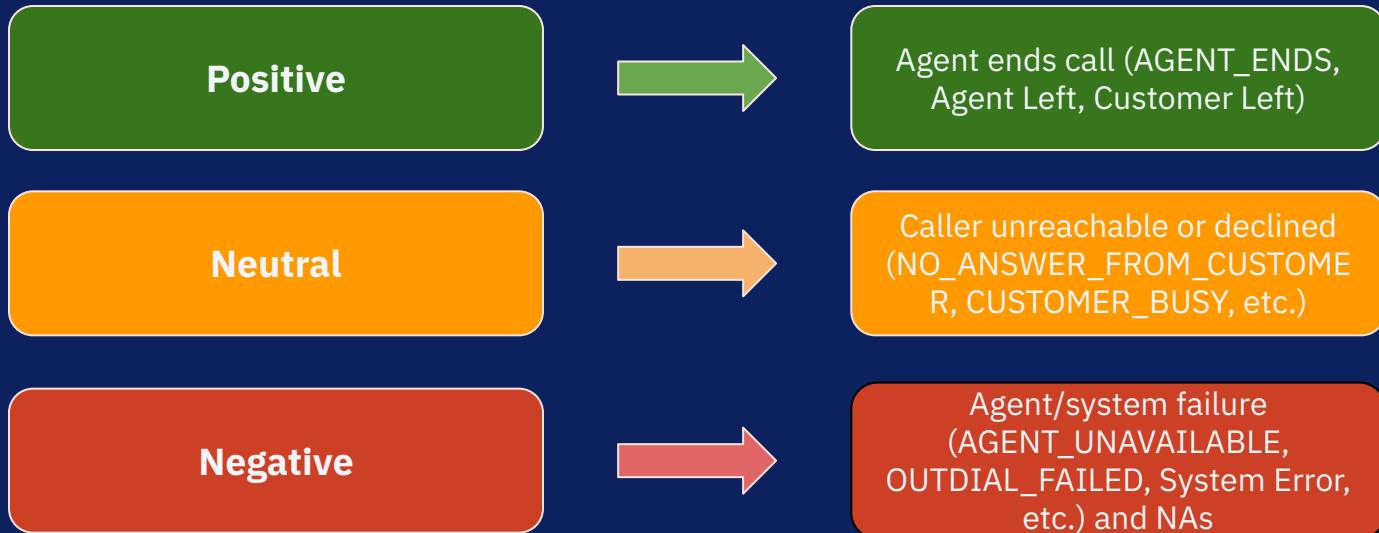
Even though the trafficking voicemail path performs well, the broader immigration routing experience is struggling to successfully connect callers and provide legal help

Employment Menu Outcomes

Goal: Analyzed Employment calls in the CAR dataset after March 16, 2025 to classify outcomes as positive, neutral, or negative.

Methodology:

Main step: Classification of the calls, using **case_when** function on the **termination_reason** variable



Employment Menu Outcomes

Data Issue:

Expected related menus under Employment: Clinic Voicemail Transfer, Employment SP, Unemployment, Wages, and Workers Comp, but all appeared as “N/A” in the CAR dataset after March 16, 2025.

Key stats:

Total number of calls: 4032

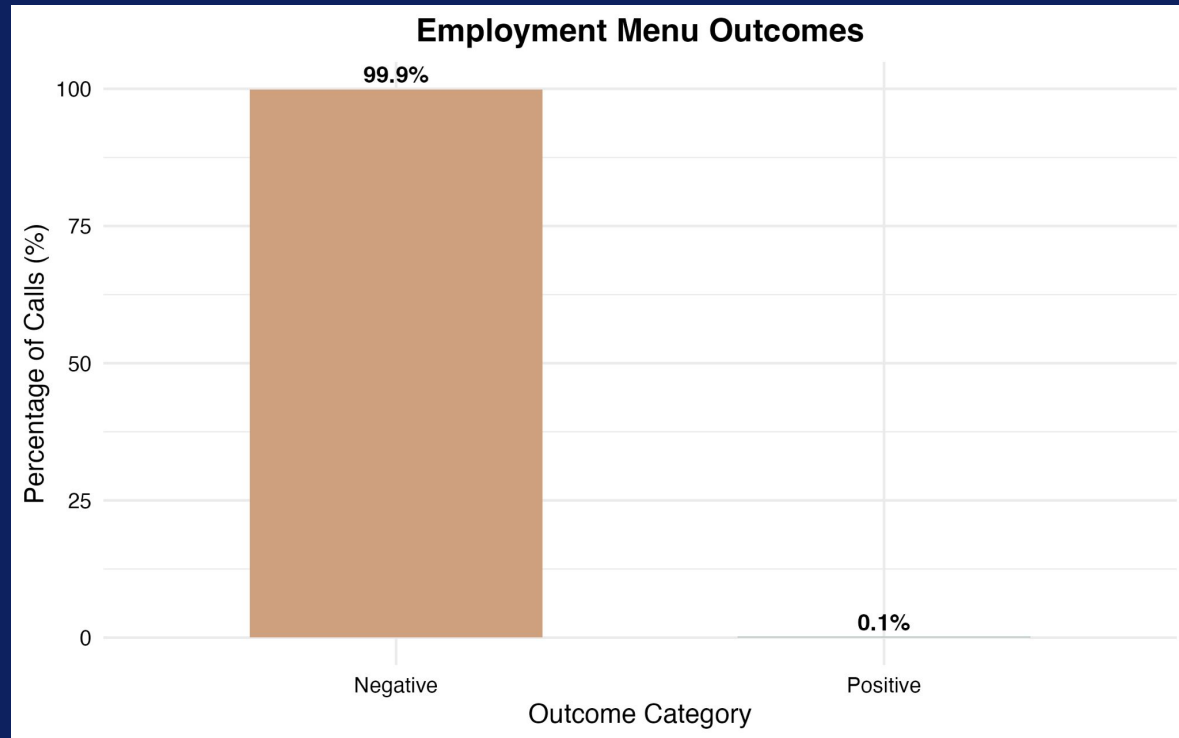
Positive calls: 5

Negative calls: 4027

Employment Menu Outcomes

Takeaways:

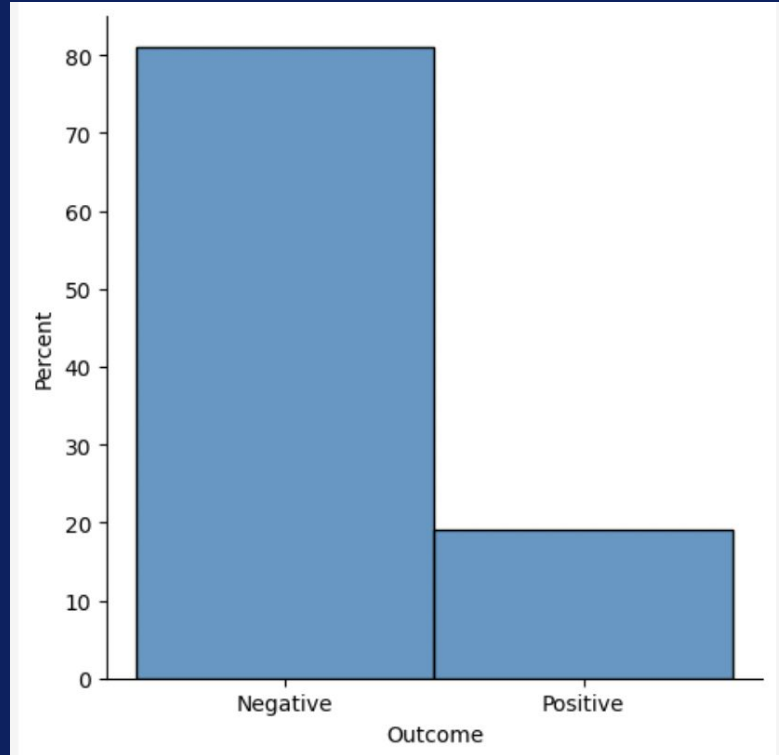
- Almost all Employment calls ($\approx 99.9\%$) in the CAR dataset remain unidentified, indicating that termination details were not properly captured for most cases.
- Only 0.1% of calls show a positive outcome, meaning the caller likely completed an interaction or reached an agent.
- No neutral calls were recorded, suggesting missing or incomplete metadata rather than true absence of these cases.



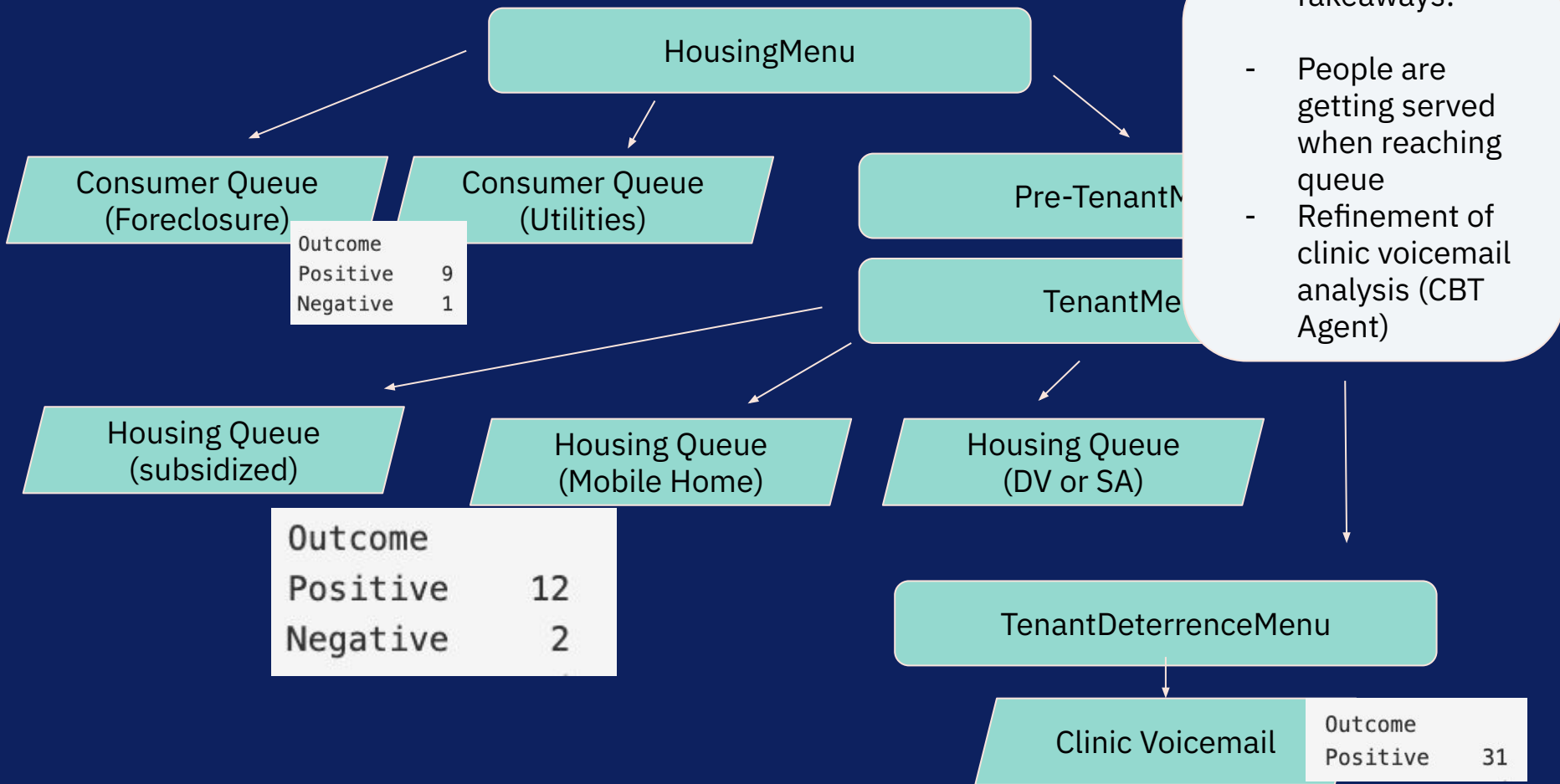
Housing Menu Outcomes

- In general, there were ~80% negative call Outcomes and ~20% positive call outcomes
- There were no neutral outcomes found
- Note: all 'customer left' outcomes were recorded as negative unless there was an 'agent left' activity name

Outcome	
Negative	234
Positive	55



Housing Menu Outcomes by Queue



Family Menu Outcomes

Outcome Breakdown: Family Menu

Outcome Type	N	%
Negative	5153	67.47
Neutral	1601	20.96
Postive	884	11.57

Methods

- Collect all contact session IDs that visited family menu
- Identify final menu each ID traversed to
- Keep ID that ended at Family/related menus (Divorce/Parenting, Child Support, Closed Queue, or Voicemail Transfer (7638))

Tagging Approach

POSITIVE: Customer left with agent, positive termination reasons from slide 2

NEUTRAL: Calls ending in menus that LAC does not service (Child Support or Simple Divorce), neutral termination reasons from slide 2

NEGATIVE: Customer left with no agent, negative termination reasons from slide 2

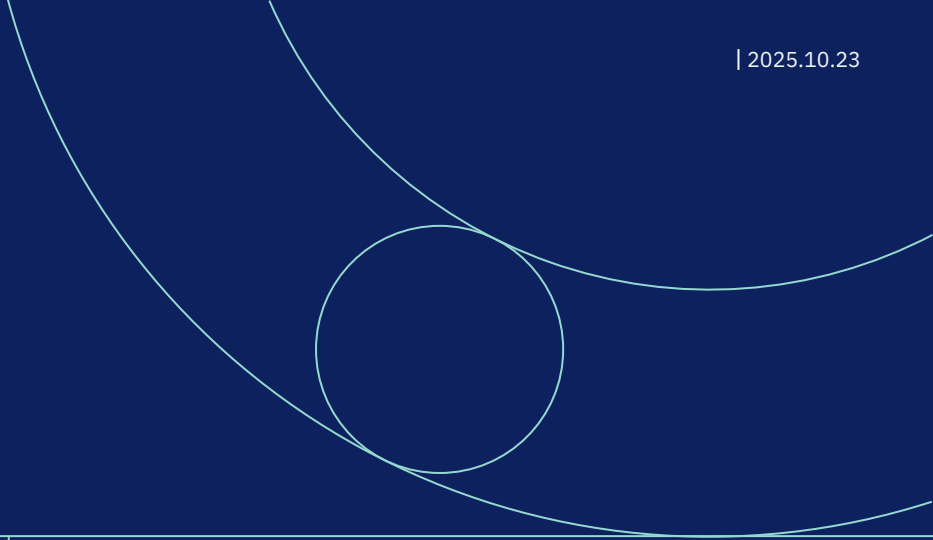
Family Menu Outcomes

Outcome Breakdown: Family Menu

Outcome Type	N	%
Negative	5153	67.47
Neutral	1601	20.96
Postive	884	11.57

Findings & Concerns

- Large volume of neutral outcomes stemming from services that LAC fails to offer (child support and simple divorce) – callers aren't being helped, but they weren't supposed to be anyways.
- High overall call volume likely contributing to large share of negative outcomes (more callers = more unable to be helped).
- Issue: case when logic resulted in some sessions being tagged as both neutral and positive, so positive outcomes likely underrepresented.
 - Family Menu also redirects to more queues/submenus than terminal points, so difficult to categorize outcomes without overgeneralization.



Thank you!

Appendix

Legal Menu Outcomes

Identifying Positive Outcomes for Calls in Queues (ADAPT, Consumer, Criminal Records)

```
def successful_call(df, id_col = 'Call ID', term_col='Termination Reason', term_val1='Customer Left',
                  term_val2 = 'Agent Left', agent_col='Agent Name'):

    # Checks if row has 'Agent Left' as termination reason, or 'Customer Left as termination reason and a non-NaN agent name
    mask = (
        (df[term_col].isin([term_val1]) & ~df[agent_col].isna()) | df[term_col].isin([term_val2])
    )

    # Filtering dataframe for rows where conditions are met
    # Storing 'Call IDs' as an array
    qualifying_calls = df.loc[mask, id_col].unique()

    # Using array to filter dataframe for those
    valid_calls = df.loc[df[id_col].isin(qualifying_calls), id_col].dropna().unique()

    return valid_calls
```

```
# Filtering dataframe to only include Contact Session IDs of calls that have 'ADAPTQueue' as an activity name
adapt_calls = menu_selection.loc[menu_selection['Activity Name'] == 'ADAPTQueue', 'Contact Session ID']

adapt_queue = menu_selection.loc[menu_selection['Contact Session ID'].isin(adapt_calls), :]

# Finding total calls in ADAPT Queue
print('Total Number of Calls in ADAPT Queue:', adapt_queue['Call ID'].nunique())

# Finding number of positive outcomes for ADAPT menu using successful_call function
adapt_success = successful_call(adapt_queue)

print('Number of Positive Outcomes:', len(adapt_success))
```

Legal Menu Outcomes

Identifying Negative Outcomes for Calls in Queues (ADAPT, Consumer, Front Desk Queue)

```
# Filtering ADAPT dataframe to exclude Call IDs of calls with positive outcomes
adapt_unsuccessful = adapt_queue.loc[~adapt_queue['Call ID'].isin(adapt_success)]

# Dropping the final row for calls if the Termination Reason is NaN
last_row = adapt_unsuccessful.groupby('Call ID').cumcount(ascending=False) == 0
no_term_reason = adapt_unsuccessful['Termination Reason'].isna()
adapt_unsuccessful = adapt_unsuccessful[~(last_row & no_term_reason)]

# Grouping last Termination Reason by Call ID
adapt_unsuccessful_calls = pd.DataFrame(adapt_unsuccessful.groupby('Call ID')['Termination Reason'].last())

# Checking if Termination Reason is in list of negative outcomes
adapt_negative = adapt_unsuccessful_calls[adapt_unsuccessful_calls['Termination Reason'].isin(['Customer Left', 'Queue Timeout', 'AGENT_UNAVAILABLE',
'AGENT_BUSY', 'NO_ANSWER_FROM_AGENT', 'MEDIA_MANAGER_INTERNAL_ERROR', 'CHANNEL_FAILURE'])]

# Printing number of negative and unexplained outcomes
print('Number of Negative Outcomes:', len(adapt_negative))
print('Unexplained Outcomes:', len(adapt_unsuccessful_calls) - len(adapt_negative))
```

✓ 0.0s

Legal Menu Outcomes

Identifying Outcomes for Criminal Records Voicemail Transfer

```
# Filtering dataframe to only include Contact Session IDs of calls that have 'CriminalRecordsVoicemailTransfer' as an activity name
criminal_calls = menu_selection.loc[menu_selection['Activity Name'] == 'CriminalRecordsVoicemailTransfer', 'Contact Session ID']

criminal_queue = menu_selection.loc[menu_selection['Contact Session ID'].isin(criminal_calls), :]

# Finding total calls in Criminal Records Menu
print('Total Number of Calls in Criminal Records Menu:', criminal_queue['Call ID'].nunique())

# Finding number of positive outcomes for Criminal Records Voicemail Transfer
crim_success = criminal_queue[criminal_queue['Termination Reason'].isin(['Agent Left'])]

print('Number of Positive Outcomes:', crim_success['Call ID'].nunique())
```

0] ✓ 0.0s

Total Number of Calls in Criminal Menu: 709
Number of Positive Outcomes: 602

```
# Filtering Criminal Records dataframe to exclude Call IDs of calls with positive outcomes
unsuccessful_criminal = criminal_queue.loc[~criminal_queue['Call ID'].isin(crim_success['Call ID'])]

# Dropping the final row for calls if the Termination Reason is NaN
last_row = unsuccessful_criminal.groupby('Call ID').cumcount(ascending=False) == 0
no_term_reason = unsuccessful_criminal['Termination Reason'].isna()
unsuccessful_criminal = unsuccessful_criminal[~(last_row & no_term_reason)]

# Grouping last Termination Reason by Call ID
crim_neutral_calls = pd.DataFrame(unsuccessful_criminal.groupby('Call ID')['Termination Reason'].last())

# Checking if Termination Reason is in list of negative outcomes
crim_neutral = crim_neutral_calls[crim_neutral_calls['Termination Reason'].isin(['Customer Left'])]

# Printing number of neutral and unexplained outcomes
print('Number of Neutral Outcomes:', len(crim_neutral))
print('Number of Unexplained Outcomes:', len(crim_neutral_calls) - len(crim_neutral))
```

5] ✓ 0.0s

Legal Menu Outcomes

Aggregating Outcomes Across Legal Menu 1 & 2

```
# Finding overall number & proportion of positive outcomes for Legal Menu (1 & 2)
positive_outcomes_overall = len(adapt_success) + len(consumer_success) + len(crim_success) + len(front_desk_success)
```

```
print('Overall Legal Menu Calls with a Positive Outcome:', positive_outcomes_overall)
print('Proportion of Overall Calls:', round(positive_outcomes_overall / menu_selection['Call ID'].nunique(), 4))
```

✓ 0.0s

Overall Legal Menu Calls with a Positive Outcome: 2073
Proportion of Overall Calls: 0.0624

```
# Finding overall number & proportion of negative outcomes for Legal Menu (1 & 2)
negative_outcomes_overall = len(adapt_negative) + len(consumer_negative) + len(negative_fd_calls)
```

```
print('Overall Legal Menu Calls with a Negative Outcome:', negative_outcomes_overall)
print('Proportion of Overall Calls:', round(negative_outcomes_overall / menu_selection['Call ID'].nunique(), 4))
```

✓ 0.0s

Overall Legal Menu Calls with a Negative Outcome: 93
Proportion of Overall Calls: 0.0028

Call Outcomes for Legal Menu 2

```
# Finding overall number of positive outcomes for Legal Menu 2
positive_outcomes2 = len(adapt_success) + len(consumer_success) + len(crim_success)
```

```
print('Legal Menu 2 Calls with a Positive Outcome:', positive_outcomes2)
```

```
# Proportion including calls that terminate in another menu
print('Proportion of Overall Legal Menu 2 Calls:', round(positive_outcomes2/ legal_menu2_calls['Contact Session ID'].nunique(), 3))
```

```
# Proportion of calls that terminate in Legal Menu 2
print('Proportion of Calls Terminating in Legal Menu 2:', round(positive_outcomes2/ terminal_node_calls, 3) )
```

✓ 0.0s

Legal Menu 2 Calls with a Positive Outcome: 1279
Proportion of Overall Legal Menu 2 Calls: 0.04
Proportion of Calls Terminating in Legal Menu 2: 0.563

Immigration Menu Outcomes

Extract and clean Immigration Menu call sessions

```
# get contact session IDs where flow_name == "LegalImmigrationMenu"
immigration_id <- car |>
  filter(flow_name == "LegalImmigrationMenu") |>
  pull(contact_session_id) |>
  unique()

# filter car by all rows belonging to those session IDs
immigration_calls <- car |>
  filter(contact_session_id %in% immigration_id)

# show # of unique contact_session_id
length(unique(immigration_calls$contact_session_id))

# count termination reasons and combine NAs, RONA Timer Expired
immigration_calls_clean <- immigration_calls |>
  mutate(
    termination_reason = trimws(termination_reason),
    termination_reason = case_when(
      is.na(termination_reason) ~ "NA",
      termination_reason %in% c("NA", "N/A") ~ "NA",
      termination_reason %in% c("RONA Timer Expired", "RONA_TIMER_EXPIRED") ~ "RONA Timer Expired",
      TRUE ~ termination_reason
    )
  )
```

Immigration Menu Outcomes

Separate Immigration Menu calls into two paths, count unique call sessions and identify the final termination outcome using the latest activity timestamp for each path

```
# CASE 1: Trafficking Voicemail Transfer ----
case1 <- immigration_calls_clean |>
  filter(queue_name == "Trafficking Voicemail Transfer" | activity_name == "TraffickingVoicemailTransfer")

# extract unique ids
case1_id <- case1 |>
  pull(contact_session_id) |>
  unique()

# show # of unique contact_session_id
length(case1_id)

# count termination reasons
case1_termination <- case1 |>
  group_by(contact_session_id) |>
  slice_max(activity_start_timestamp, n = 1, with_ties = FALSE) |> # latest row only
  ungroup() |>
  count(termination_reason, sort = TRUE)

# CASE 2: Immigration or Immigration SP (all other outcomes, not trafficking voicemail) ----
case2 <- immigration_calls_clean |>
  filter(!(contact_session_id %in% case1_id))

# show # of unique contact_session_id
length(unique(case2$contact_session_id))

# count termination reasons
case2_termination <- case2 |>
  group_by(contact_session_id) |>
  slice_max(activity_start_timestamp, n = 1, with_ties = FALSE) |> # latest row only
  ungroup() |>
  count(termination_reason, sort = TRUE)
```

Immigration Menu Outcomes

Combine both routing paths and label each call as Positive, Neutral, or Negative

```
# Classification ----
# prepare final outcomes from Case 1 & Case 2
final_outcomes <- bind_rows(
  # Case 1 (keep all)
  case1 |>
    group_by(contact_session_id) |>
    slice_max(activity_start_timestamp, n = 1, with_ties = FALSE) |>
    ungroup() |>
    mutate(case = "Trafficking Voicemail Transfer"),

  # Case 2 (filter out NA termination_reason)
  case2 |>
    group_by(contact_session_id) |>
    slice_max(activity_start_timestamp, n = 1, with_ties = FALSE) |>
    ungroup() |>
    filter(!is.na(termination_reason) & termination_reason != "NA") |>
    mutate(case = "Other Immigration Routes")
) |>
# add agent involvement indicator
mutate(connected_to_agent = !is.na(agent_name) & agent_name != "N/A")
```

```
# classify call outcomes: POSITIVE/NEUTRAL/NEGATIVE
# POSITIVE: Caller connects with an agent and the call ends normally
# NEGATIVE: Caller does not reach an agent due to abandonment or system/agent failure
# NEUTRAL: Caller receives information without agent support or contact is unreachable/out-of-scope

classified_outcomes <- final_outcomes |>
  mutate(
    outcome = case_when(
      # POSITIVE
      termination_reason == "Agent Left" ~ "Positive",
      termination_reason == "Customer Left" & connected_to_agent ~ "Positive",

      # NEGATIVE
      termination_reason %in% c("AGENT_UNAVAILABLE", "AGENT_BUSY",
                                "NO_ANSWER_FROM_AGENT", "MEDIA_MANAGER_INTERNAL_ERROR",
                                "CHANNEL_FAILURE") ~ "Negative",
      termination_reason == "Customer Left" & !connected_to_agent ~ "Negative",

      # NEUTRAL
      TRUE ~ "Neutral"
    )
  )
```

Housing Menu Outcomes

Finding call outcomes for all calls that navigate to the housing menu (289 calls)

```
df_house['is_positive'] = (  
    (df_house['Step Category'] == 'Agent Name') |  
    (df_house['Activity'] == 'Agent Left') |  
    (df_house['Activity'] == 'AGENT_ENDS')  
)
```

✓ 0.0s

```
df_house['is_neutral'] = (  
    (df_house['Activity'] == 'NO_ANSWER_FROM_CUSTOMER') |  
    (df_house['Activity'] == 'CUSTOMER_BUSY') |  
    (df_house['Activity'] == 'MAX_CALLBACK_RETRY_LIMIT_REACHED') |  
    (df_house['Activity'] == 'Participant Invite timer expired') |  
    (df_house['Activity'] == 'CUSTOMER_UNAVAILABLE') |  
    (df_house['Activity'] == 'USER_DECLINED') |  
    (df_house['Activity'] == 'USER_BUSY') |  
    (df_house['Activity'] == 'NO_ANSWER_USER')  
)
```

✓ 0.0s

```
df_house['is_negative'] = (  
    (df_house['Activity'] == 'AGENT_UNAVAILABLE') |  
    (df_house['Activity'] == 'AGENT_BUSY') |  
    (df_house['Activity'] == 'MAX_CALLBACK_RETRY_LIMIT_REACHED') |  
    (df_house['Activity'] == 'NO_ANSWER_FROM_AGENT') |  
    (df_house['Activity'] == 'OUTDIAL_FAILED') |  
    (df_house['Activity'] == 'MEDIA_MANAGER_INTERNAL_ERROR') |  
    (df_house['Activity'] == 'CHANNEL_FAILURE')  
)
```

```
# Group by session and collect all applicable tags  
session_flags = df_house.groupby('Contact Session ID').agg({  
    ... 'is_positive': 'max', ... # True if any row in session is positive  
    ... 'is_neutral': 'max', ... # True if any row is neutral  
    ... 'is_negative': 'max', ... # True if any row is negative  
})
```

Create a list of outcomes for each session

```
def collect_outcomes(row):  
    ... outcomes = []  
    ... if row['is_positive']:  
    ...     outcomes.append('Positive')  
    ... if row['is_neutral']:  
    ...     outcomes.append('Neutral')  
    ... if row['is_negative']:  
    ...     outcomes.append('Negative')  
    ... if not outcomes:  
    ...     outcomes.append('Negative')  
    ... return ','.join(outcomes)
```

```
session_flags['Outcome'] = session_flags.apply(collect_outcomes, axis=1)
```

✓ 0.0s

Keep Outcome as a DataFrame

```
session_outcome = session_flags[['Outcome']].copy()
```

Merge back to original DataFrame

```
df_house = df_house.merge(  
    ... session_outcome,  
    ... how='left',  
    ... left_on='Contact Session ID',  
    ... right_index=True  
)
```

Optional: drop temporary flags

```
df_house.drop(columns=['is_positive', 'is_neutral', 'is_negative'], inplace=True)
```

Housing Menu Queue Outcomes

Finding call outcomes for all calls that were placed in a queue (55 calls)

```
queue_name = df_house[df_house['Step Category'] == "Queue Name"]
```

✓ 0.0s

```
consumer = []  
housing = []  
voicemail = []
```

```
# Loop through each call journey  
for session_id, group in queue_name.groupby('Contact Session ID'):
```

```
    # Check for Consumer queues  
    if group['Activity'].isin(["Consumer", "Consumer SP"]).any():  
        consumer.append(df_house.loc[session_id, :])
```

```
    # Check for Housing queues  
    if group['Activity'].isin(["Housing", "Housing SP"]).any():  
        housing.append(df_house.loc[session_id, :])
```

```
    # Check for Clinic Voicemail  
    if group['Activity'].eq("Clinic Voicemail Transfer").any():  
        voicemail.append(df_house.loc[session_id, :])
```

```
df_consumer = pd.concat(consumer)  
df_housing = pd.concat(housing)  
df_voicemail = pd.concat(voicemail)
```

```
df_consumer['is_positive'] = (  
    (df_consumer['Step Category'] == 'Agent Name') |  
    (df_consumer['Activity'] == 'Agent Left') |  
    (df_consumer['Activity'] == 'AGENT_ENDS')  
)
```

```
df_consumer['is_neutral'] = (  
    (df_consumer['Activity'] == 'NO_ANSWER_FROM_CUSTOMER') |  
    (df_consumer['Activity'] == 'CUSTOMER_BUSY') |  
    (df_consumer['Activity'] == 'MAX_CALLBACK_RETRY_LIMIT_REACHED') |  
    (df_consumer['Activity'] == 'Participant Invite timer expired') |  
    (df_consumer['Activity'] == 'CUSTOMER_UNAVAILABLE') |  
    (df_consumer['Activity'] == 'USER_DECLINED') |  
    (df_consumer['Activity'] == 'USER_BUSY') |  
    (df_consumer['Activity'] == 'NO_ANSWER_USER')  
)
```

```
df_consumer['is_negative'] = (  
    (df_consumer['Activity'] == 'AGENT_UNAVAILABLE') |  
    (df_consumer['Activity'] == 'AGENT_BUSY') |  
    (df_consumer['Activity'] == 'MAX_CALLBACK_RETRY_LIMIT_REACHED') |  
    (df_consumer['Activity'] == 'NO_ANSWER_FROM_AGENT') |  
    (df_consumer['Activity'] == 'OUTDIAL_FAILED') |  
    (df_consumer['Activity'] == 'MEDIA_MANAGER_INTERNAL_ERROR') |  
    (df_consumer['Activity'] == 'CHANNEL_FAILURE')  
)
```

```
# Group by session and collect all applicable tags  
session_flags_con = df_consumer.groupby('Contact Session ID').agg({  
    'is_positive': 'max', # True if any row in session is positive  
    'is_neutral': 'max', # True if any row is neutral  
    'is_negative': 'max', # True if any row is negative  
})
```

```
# Create a list of outcomes for each session
```

```
def collect_outcomes(row):  
    outcomes = []  
    if row['is_positive']:  
        outcomes.append('Positive')  
    if row['is_neutral']:  
        outcomes.append('Neutral')  
    if row['is_negative']:  
        outcomes.append('Negative')  
    if not outcomes:  
        outcomes.append('Negative')  
    return ', '.join(outcomes)
```

```
session_flags_con['Outcome'] = session_flags_con.apply(collect_outcomes, axis=1)
```

✓ 0.0s

```
session_outcome_con = session_flags_con[['Outcome']].copy()
```

```
# Merge back to original DataFrame
```

```
df_consumer = df_consumer.merge(  
    session_outcome_con,  
    how='left',  
    left_on='Contact Session ID',  
    right_index=True  
)
```

```
# Optional: drop temporary flags
```

```
df_consumer.drop(columns=['is_positive', 'is_neutral', 'is_negative'], inplace=True)  
session_outcome_con
```

✓ 0.0s

Employment Menu Queue

Filtering CAR dataset focusing on
Employment Menu



```
car_employment <- car |>
  filter(
    flow_name == "LegalEmploymentMenu" | activity_name == "EmploymentMenu",
    activity_start_timestamp >= as.POSIXct("2025-03-16")
  ) |>
  reframe(
    .by = contact_session_id,
    termination_reason = termination_reason,
    activity_start_timestamp = activity_start_timestamp,
    activity_name = activity_name
  ) |>
  distinct()
```

Classifying Employment Menu
Calls by positive, neutral and
negative outcomes



```
car_employment <- car_employment |>
  mutate(
    outcome = case_when(
      termination_reason %in% c("Agent Left", "AGENT_ENDS", "Customer Left") ~ "Positive",
      termination_reason %in% c(
        "Queue Timeout", "CUSTOMER_UNAVAILABLE", "MAX_CALLBACK_RETRY_LIMIT_REACHED",
        "NO_ANSWER_FROM_CUSTOMER", "NO_ANSWER_CUSTOMER", "NO_ANSWER_USER",
        "CUSTOMER_BUSY", "USER_UNAVAILABLE", "USER_BUSY", "USER_DECLINED",
        "Participant Invite timer expired", "RONA_TIMER_EXPIRED", "RONA Timer Expired",
        "CONTACT_CALLBACK_IN_PROGRESS"
      ) ~ "Neutral",
      termination_reason %in% c(
        "NO_ANSWER_FROM_AGENT", "AGENT_UNAVAILABLE", "AGENT_BUSY",
        "OUTDIAL_FAILED", "MEDIA_MANAGER_INTERNAL_ERROR", "CHANNEL_FAILURE",
        "System Error", "System disconnected the contact"
      ) ~ "Negative",
      termination_reason %in% c("NA", "N/A") | is.na(termination_reason) ~ "Negative",
      TRUE ~ "Unidentified"
    )
  )
```

Employment Menu Queue

Housing Menu Calls summarized
by outcome category



```
employment_summary <- car_employment |>
  count(outcome) |>
  mutate(percentage = round(n / sum(n) * 100, 1))
```

Creating plot to show the
categorization of Housing Menu
Calls arranged by outcome
category



```
employment_plot <- ggplot(employment_summary, aes(x = reorder(outcome, -percentage),
                                                    y = percentage,
                                                    fill = outcome)) +
  geom_col(width = 0.6) +
  geom_text(aes(label = paste0(percentage, "%"),
                    vjust = -0.4,
                    color = "black",
                    fontface = "bold",
                    size = 4)) +
  scale_fill_manual(values = c(
    "Positive" = "#5C8D89",      # muted green
    "Neutral" = "#B1B6BE",      # soft gray
    "Negative" = "#CFA07E",     # muted brown/taupe
    "Unidentified" = "#708090"  # slate gray
  )) +
  labs(
    title = "Employment Menu Outcomes",
    x = "Outcome Category",
    y = "Percentage of Calls (%)"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    axis.text = element_text(color = "black"),
    legend.position = "none"
  )
```

Family Menu - Filter CAR data

```
car <- read_csv('data/combined_data.csv') |> janitor::clean_names() |>
  mutate(activity_start_timestamp = ymd_hms(activity_start_timestamp),
         across(where(is.character), ~ na_if(.x, "N/A")),
         hour = hour(activity_start_timestamp)) |>
  filter(activity_start_timestamp >= '2025-03-16 00:00:00')

# only looking at contact session ids that visited the family menu after March 16, 2025
family_ids <-
  car |>
  filter(ep_name == 'Legal Family Menu Telephony EP') |>
  distinct(contact_session_id) |>
  pull(contact_session_id)

family_sessions <-
car |>
  filter(contact_session_id %in% family_ids) |>
  arrange(contact_session_id, activity_start_timestamp)
```

Family Menu - Collect Contact Session IDs

```
# grab final menu traversed to
family_sessions <-
family_sessions |>
  group_by(contact_session_id) |>
  mutate(
    last_menu = last(activity_name, order_by = activity_start_timestamp, na_rm = TRUE),
    last_reason = last(termination_reason, order_by = activity_start_timestamp, na_rm = TRUE)
  ) |>
  ungroup()

# list of where calls involving family menu ended up...filter out those with
# nothing to do with family menu/offshoots
final_menus <-
family_sessions |>
  count(last_menu, sort = TRUE)

relevant_menus <- c('ClosedQueueMenu', 'ClinicVoicemailTransfer',
                    'DivorceOrParentingMenu', 'ChildSupportMenu',
                    'FamilyMenu', 'SimpleDivorceMenu')

family_sessions <-
family_sessions |>
  filter(last_menu %in% relevant_menus)
```

Family Menu - Sort termination reasons with outcome type

```
negative <- c('AGENT_UNAVAILABLE',  
             'AGENT_BUSY',  
             'NO_ANSWER_FROM_AGENT',  
             'MEDIA_MANAGER_INTERNAL_ERROR',  
             'System disconnected the contact')  
positive <- c('Agent Left', 'AGENT_ENDS')  
neutral <- c('Participant Invite timer expired',  
            'NO_ANSWER_FROM_CUSTOMER',  
            'CUSTOMER_BUSY',  
            'MAX_CALLBACK_RETRY_LIMIT_REACHED',  
            'CUSTOMER_UNAVAILABLE',  
            'USER_DECLINED',  
            'USER_BUSY',  
            'USER_UNAVAILABLE',  
            'NO_ANSWER_USER',  
            'Queue Timeout')
```

Family Menu - Encode outcomes and create table

```
# create outcomes table|
family_outcomes <-
family_sessions |>
  summarize(
    .by = contact_session_id,
    positive = any(!is.na(agent_name) & termination_reason == "Customer Left") |
      (termination_reason %in% positive)),
    neutral = any(termination_reason %in% neutral |
      str_detect(activity_name, regex("ChildSupportMenu|SimpleDivorceMenu", ignore_case = TRUE))),
    negative = any(is.na(agent_name) & termination_reason == "Customer Left" |
      (termination_reason %in% negative))
  ) |>
  mutate(
    outcome = case_when(
      neutral ~ 'Neutral',
      positive ~ 'Positive',
      negative ~ 'Negative'
    )
  ) |>
  count(outcome) |>
  mutate(
    pct = (n / sum(n)) * 100
  ) |>
  knitr::kable(
    caption = 'Outcome Breakdown: Family Menu',
    col.names = c('Outcome Type', 'N', '%'),
    digits = 2
  )
```